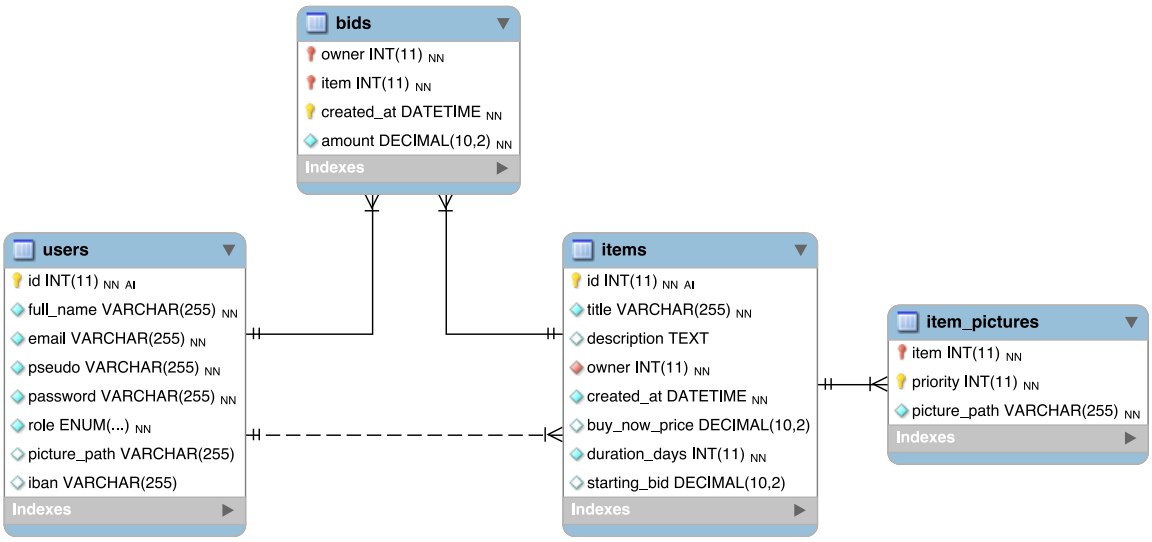
Données et validations métier

Schéma de la base de données

Le schéma de la base de données vous est imposé. Le script SQL de création se trouvera dans votre dépôt BitBucket. Aucune modification de ce schéma n'est autorisée (pas d'ajout, modification ou suppression des tables, champs, contraintes, index, vues, procédures stockées, triggers, etc).



Remarques générales :

- les colonnes marquées par `NN` sont des colonnes `NOT NULL` ;
- les colonnes marquées par `AI` sont des clés primaires autoincrémentées ;
- la taille maximale des champs de type `VARCHAR` ou `TEXT` est régie par la base de données sur base de la définition des colonnes correspondantes dans le schéma ; elle doit faire l'objet d'une validation explicite dans votre application.

Le script de création de la base de données insère plusieurs utilisateurs dans la table `users`, dont voici les *credentials* :

E-mail	Mot de passe	Rôle
`admin@epfc.eu`	`Password1,`	`admin`
`boverhaegen@epfc.eu`	`Password1,`	`user`

E-mail	Mot de passe	Rôle
`mamichel@epfc.eu`	`Password1,`	`user`
`quhouben@epfc.eu`	`Password1,`	`user`
`xapigeolet@epfc.eu`	`Password1,`	`user`

Description des tables

- La table `users` contient les **utilisateurs** du système, caractérisés par un identifiant unique (`id`), un e-mail (`email`), un mot de passe (`password`), un nom complet (`full_name`), un pseudo (`pseudo`), un rôle (`role`), le chemin éventuel d'une photo de profil (`picture_path`) et un éventuel numéro de compte (`iban`).

  Il y a deux types d'utilisateurs : les utilisateurs simples (colonne `role` à `'user'`) et les administrateurs (colonne `role` à `'admin'`).

- La table `items` contient les **annonces**, caractérisées par un identifiant unique (`id`), un titre (`title`), une description (`description`), une heure/date de création (`created_at`), une durée en jours (`duration_days`), un éventuel prix d'achat immédiat (`buy_now_price`) et une éventuelle enchère minimale (`starting_bid`).

  Une annonce est liée à un utilisateur qui est son créateur / propriétaire (colonne `owner`).

  Une annonce débute immédiatement à sa création (`created_at`). La durée en jours (`duration_days`) détermine la date/heure de fin des enchères.

  > **Note**
  >
  > Une annonce qui contient une enchère minimale (`starting_bid` non nul et différent de 0) peut recevoir des offres d'achat (enchères) de la part des utilisateurs de l'application.
  >
  > Une annonce peut éventuellement contenir un prix d'achat immédiat, auquel cas un utilisateur peut décider d'acheter immédiatement l'annonce au prix indiqué. Un achat immédiat se concrétise par la création d'une offre d'achat (enchère) dont le montant est supérieur ou égal à `buy_now_price` .
  >
  > Une annonce peut combiner les deux systèmes (enchères et achat immédiat) : dans ce cas, la première enchère qui atteint ou dépasse le prix d'achat immédiat remporte l'annonce et clôture les enchères. Si à la fin de la période d'enchères aucune enchère n'a atteint le prix d'achat immédiat, l'annonce est remportée par l'enchère la plus élevée.

- La table `bids` contient les **offres d'achat** faites par les utilisateurs sur les annonces. Une offre d'achat est caractérisée par une date/heure de création (`created_at`) et un montant (`amount`).

  Une offre d'achat est liée à un utilisateur qui est son créateur (colonne `owner`) et à une annonce sur laquelle elle a été faite (colonne `item`).

  La clé primaire de cette table est composée des colonnes `owner` , `item` et `created_at` , ce qui signifie qu'un même utilisateur peut faire plusieurs offres d'achat sur une même annonce, à des moments différents.

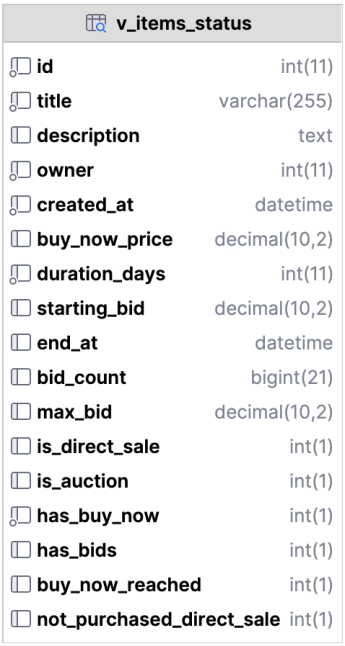
  > **Note**
  >
  > Le montant d'une offre d'achat doit toujours être supérieur au montant de l'offre la plus élevée déjà faite sur l'annonce (ou à l'enchère minimale si aucune offre n'a encore été faite).
  >
  > Il n'est pas possible de faire une offre d'achat sur une annonce clôturée (période d'enchères terminée ou prix d'achat immédiat atteint). Il n'est pas non plus possible de faire une offre d'achat sur une annonce que l'on possède soi-même ou sur une annonce qui n'a pas encore débuté.

- La table `item_pictures` contient les **photos des annonces**. Une photo est caractérisée par un chemin vers le fichier image (`picture_path`) et une priorité d'ordre d'affichage (`priority`).

  La colonne `item` contient l'identifiant de l'annonce à laquelle appartient la photo.

  Une annonce peut contenir plusieurs photos. La priorité d'affichage permet de déterminer l'ordre dans lequel les photos seront affichées dans l'application (la photo de priorité `1` sera affichée en premier, la photo de priorité `2` en second, etc).

Description de la vue `v_items_status`



La vue `v_items_status` permet d'obtenir des informations sur le statut des annonces afin de simplifier vos requêtes SQL. Elle contient, pour chaque annonce, les colonnes de la table `items` et les colonnes suivantes :

- `end_at` : la date/heure de fin des enchères de l'annonce (calculée en fonction de la date/heure de création et de la durée) ;
- `bid_count` : le nombre d'offres d'achat faites sur l'annonce ;
- `max_bid` : le montant de l'offre d'achat la plus élevée faite sur l'annonce (ou `NULL` si aucune offre n'a encore été faite) ;
- `is_direct_sale` : un booléen indiquant si l'annonce est une vente directe (achat immédiat uniquement, sans enchères) ;
- `is_auction` : un booléen indiquant si l'annonce est une enchère (avec ou sans achat immédiat) ;
- `has_buy_now` : un booléen indiquant si l'annonce possède un prix d'achat immédiat ;
- `has_bids` : un booléen indiquant si l'annonce a reçu des offres d'achat ;
- `buy_now_reached` : vaut 1 si une offre atteint ou dépasse le `buy_now_price`, indépendamment de la date de fin des enchères ;
- `not_purchased_direct_sale` : un booléen indiquant si l'annonce n'a pas été achetée via l'achat immédiat ;

Remarque

Les booléens sont représentés par des entiers : `0` pour `FALSE` et `1` pour `TRUE`.

Validations métier

Voici quelques contraintes métier complémentaires à celles imposées par le schéma de la base de données ci-dessus :

> **Attention**
>
> Les constantes de longueurs minimales ou maximales de chaînes de caractères, de dimensions des images ou de tailles de fichiers reprises ci-dessous doivent être définies dans le fichier de configuration (`dev.ini` ou `prod.ini`). Si la longueur maximale n'est pas spécifiée dans ces règles, alors c'est celle du schéma de la base de données qui doit être prise en compte.

Table `users` :

- `id` est auto-incrémenté.
- `full_name` doit avoir au minimum une longueur de 3 caractères.
- `email` doit avoir le format d'une adresse mail valide ; il doit être unique.
- `pseudo` doit avoir une longueur comprise entre 3 et 30 caractères et doit être unique.
- `password` doit contenir la version *hachée* du mot de passe de l'utilisateur. Ce dernier doit avoir au minimum une longueur de 8 caractères, doit contenir au moins un chiffre, une lettre majuscule, une lettre minuscule et un caractère non alphanumérique.
- `role` contient le rôle de l'utilisateur (`admin` , `user`).
- `iban` , si elle est remplie, doit avoir un format IBAN de forme BE99 9999 9999 9999.
- `picture_path` , si elle est remplie, doit être un chemin valide vers un fichier image existant sur le serveur. Voir la section "gestion des images" plus bas.

Table `items` :

- `id` est auto-incrémenté.
- `title` doit avoir au minimum une longueur de 3 caractères. Le titre doit être unique pour une même `owner` .
- `description` , si elle est remplie, doit avoir au minimum une longueur de 3 caractères.
- les colonnes `starting_bid` et `buy_now_price` sont des `DECIMAL(10,2)` (10 chiffres au total, dont 2 après la virgule).
- la colonne `starting_bid` , si elle est remplie, doit être supérieure à `0` et dans ce cas, c'est une offre sur laquelle on peut enchérir. Si elle n'est pas remplie ou égale à `0` , cela signifie que l'annonce ne peut pas recevoir d'enchères mais seulement un achat immédiat.
- la colonne `buy_now_price` , si elle est remplie, doit être strictement supérieure à `0` et strictement supérieur à `starting_bid` (si `starting_bid` n'est pas nulle). Une offre dont

`starting_bid` est nulle doit avoir un `buy_now_price`.

- `duration_days` doit être un entier compris entre `1` et `365`.

Table `bids`

- La clé primaire est composée des colonnes `owner`, `item` et `created_at`.
- `amount`, de type `DECIMAL(10,2)`, doit être strictement supérieur au montant de l'offre la plus élevée déjà faite sur l'annonce (ou supérieure à `starting_bid` si aucune offre n'a encore été faite, ou égale à `buy_now_price` dans le cadre d'un achat immédiat).
- `created_at` doit être une date/heure comprise entre la date/heure de création de l'annonce (`items.created_at`) et sa clôture (durée terminée ou prix d'achat immédiat atteint).

Table `item_pictures` :

- La clé primaire est composée des colonnes `item` et `priority`.
- `picture_path` doit être un chemin valide vers un fichier image existant sur le serveur. Voir la section "gestion des images" plus bas.
- `priority` doit être un entier strictement supérieur à `0`. Deux photos d'une même annonce ne peuvent pas avoir la même priorité.

Validation et gestion des images

Dans ce projet, les images (photos de profil des utilisateurs et photos des annonces) doivent être gérées de la manière suivante :

Votre application doit permettre l'upload de fichiers images.

Les formats supportés doivent être JPEG, PNG, GIF et WEBP. La taille des fichiers uploadés ne doit pas dépasser 5MB.

Pour optimiser l'affichage des images dans l'application, les images uploadées doivent être redimensionnées et compressées avant d'être stockées sur le serveur. Votre système créera deux versions au format JPEG de chaque image uploadée : une version principale (max 1080px/1080px) et une vignette (max 360px/360px) tout en gardant les proportions originales.

Les images doivent être stockées dans un répertoire spécifique sur le serveur (`uploads/items/<ID>/<NAME>.jpg` ou `uploads/users/<ID>/<NAME>.jpg`), et le chemin vers chaque image principale doit être enregistré dans la base de données (colonnes `picture_path` des tables `users` et `item_pictures`). `ID` désigne la clé de l'entité correspondante et `NAME` est, par exemple, le nom original du fichier uploadé.

Les vignettes seront au même endroit que les images principales, avec le suffixe `_thumbnail` avant l'extension du fichier (par exemple : `uploads/items/<ID>/<NAME>_thumbnail.jpg`).

Dans l'interface graphique de votre application, les images principales seront utilisées pour l'affichage détaillé (page d'une annonce, profil utilisateur), tandis que les vignettes seront utilisées pour les listes (liste des annonces, liste des utilisateurs).

D'un point de vue technique, vous devez utiliser l'extension GD de PHP pour le traitement des images. Internet regorge de documentation à ce sujet, notamment sur le site officiel de PHP.

Remarque : les contraintes de tailles maximales des fichiers uploadés et les dimensions maximales des images transformées doivent être définies dans le fichier de configuration (`dev.ini` ou `prod.ini`).

Droits d'accès

Voici les règles qui régissent les droits d'accès :

- Tous les utilisateurs, y compris les utilisateurs invités (*guest*), peuvent parcourir la liste des annonces actives (non clôturées) et consulter le détail d'une annonce.
- Tant qu'une annonce n'a aucune offre d'achat, son propriétaire peut la modifier ou la supprimer. Dès qu'une offre d'achat a été faite sur une annonce, celle-ci ne peut plus être modifiée ou supprimée par son propriétaire.
- Tous les utilisateurs non invités peuvent faire des offres d'achat sur une annonce active (et non clôturée) qui ne leur appartient pas.
- Tout utilisateur non invité peut modifier son profil (informations personnelles, mot de passe, photo de profil, numéro de compte).
- Tout utilisateur non invité peut consulter la liste de ses achats (annonces qu'il a remportées) et des annonces qu'il a créées.
- Dans cette première itération, on ne tient pas compte du rôle `'admin'` (on considère qu'il a les mêmes droits d'accès que le rôle `'user'`).

Suppressions en cascade

Les suppressions, lorsqu'elles sont autorisées, doivent se faire en cascade. Cela signifie que lorsqu'on supprime un enregistrement d'une table, on devra supprimer en cascade tous les enregistrements des autres tables qui référencent cet enregistrement à travers une clé étrangère.

Aucune contrainte `ON DELETE CASCADE` n'est définie en base de données : la cascade doit être assurée par le code applicatif.

Par exemple, si l'on supprime une annonce, il faudra également supprimer ses images.

Dès lors, toute suppression en cascade doit être déclenchée moyennant une demande de confirmation de l'utilisateur qui précise ce qui va être supprimé.

Import et export des données

Le contrôleur `ControllerSetup` donné permet de gérer l'import et l'export des données de votre application via les actions suivantes :

- `setup/install` installe les données de base (SQL et images) et adapte les dates pour qu'elles soient dans un passé proche;
- `setup/export` exporte vos propres données SQL dans `/database/prwb_2526_xyy_private_dump.sql` et vos propres images dans `/database/prwb_2526_xyy_private_images_backup/`;
- `setup/restore` restaure ce backup privé (SQL et images).

Attention à ne pas stocker de binaires (images,...) dans votre dépôt git à l'exception des binaires déjà présents.